

FEUILLE DE ROUTE

Licence GL → Ingénieur MLOps

Parcours progressif sur plusieurs années

Objectif : devenir un ingénieur logiciel solide pendant la Licence, puis se spécialiser progressivement en Machine Learning et MLOps en Master.

Page 2 — Notre stratégie

Tu es actuellement en Licence Génie Logiciel. Nous allons donc utiliser la Licence pour construire des fondations très solides, sans chercher à apprendre immédiatement toutes les technologies MLOps.

Axe principal : Python + logique de programmation + algorithmique.

Axe parallèle : Git/GitHub → Linux/Bash → SQL → Data → DevOps → Cloud → MLOps.

Les autres langages étudiés dans ta Licence (C, C++, Java, C#, PHP, JavaScript, etc.) restent utiles pour ta formation de développeur. Ils n'ont simplement pas besoin d'être tous maîtrisés au même niveau que Python.

Principe : apprendre peu de choses à la fois, pratiquer beaucoup, construire des projets et rechercher l'autonomie sans dépendre de l'IA.

Page 3 — Vue d'ensemble

Le parcours complet est organisé en grandes phases :

- **Phase 1 — Fondations** : Python, algorithmique, structures de données.
- **Phase 2 — Outils d'ingénieur** : Git/GitHub, Linux/Bash, SQL.
- **Phase 3 — Data** : NumPy, Pandas, visualisation, statistiques.
- **Phase 4 — Machine Learning** : concepts ML, Scikit-learn, projets.
- **Phase 5 — Deep Learning** : PyTorch et bases de l'entraînement moderne.
- **Phase 6 — Software Engineering** : APIs, FastAPI, tests, packaging, architecture.
- **Phase 7 — DevOps** : Docker, CI/CD, réseaux, sécurité.
- **Phase 8 — Cloud** : AWS/Azure/GCP, services cloud et déploiement.
- **Phase 9 — MLOps** : MLflow, DVC, pipelines, registry, serving, monitoring.
- **Phase 10 — Production avancée** : Kubernetes, Terraform, observabilité, systèmes distribués.
- **Phase 11 — Spécialisation** : LLM/GenAI, GPU et MLOps avancé.

Page 4 — Phase 1 : Python + algorithmique

Objectif : devenir autonome en programmation.

- Syntaxe Python et types
- Conditions et boucles
- Listes, dictionnaires, tuples, sets
- Strings
- Fonctions et portée des variables
- Exceptions
- Fichiers
- Modules et packages
- Programmation orientée objet
- Structures de données
- Complexité algorithmique
- Recherche, tri, récursivité et résolution de problèmes
- Projets Python

Critère de validation : résoudre des problèmes de difficulté croissante sans demander immédiatement à l'IA d'écrire le code.

Page 5 — Phase 2 : Git, Linux et SQL

Git/GitHub — premier module parallèle

- Repository, staging, commits et historique
- Branches, merge et conflits
- Remote, clone, push, pull, fetch
- GitHub, Pull Requests, Issues et README
- .gitignore et bonnes pratiques
- GitHub Actions plus tard pour la CI/CD

Linux/Bash

- Terminal, fichiers, permissions
- Processus, services et logs
- SSH et réseau de base
- Scripts Bash

SQL

- SELECT, filtres, agrégations
- JOIN, sous-requêtes
- Indexes et transactions
- Conception de bases de données

Page 6 — Phase 3 : Data

Une fois Python suffisamment solide, nous passons à la manipulation de données.

- **NumPy** : tableaux, opérations vectorisées, calcul numérique.
- **Pandas** : DataFrames, nettoyage, filtrage, groupements et transformation.
- **Visualisation** : Matplotlib et lecture de graphiques.
- **Statistiques** : moyenne, variance, distributions, corrélation et notions utiles au ML.
- Formats et flux de données : CSV, JSON, Parquet, APIs.

Cette phase prépare directement le terrain pour le Machine Learning.

Page 7 — Phase 4 : Machine Learning

Objectif : comprendre ce qu'est un modèle ML, comment on l'entraîne et comment on l'évalue.

- Régression et classification
- Clustering
- Train / validation / test
- Features et labels
- Préprocessing
- Overfitting et underfitting
- Cross-validation
- Data leakage
- Hyperparamètres
- Accuracy, precision, recall, F1, ROC-AUC et métriques adaptées
- Scikit-learn
- Projets ML complets

Page 8 — Phase 5 : Deep Learning

Framework principal : PyTorch.

- Tenseurs
- Datasets et DataLoaders
- Neural networks
- Forward / backward propagation
- Loss functions
- Optimizers
- Training loops
- Validation
- GPU
- Sauvegarde et chargement de modèles

Le but n'est pas de devenir chercheur en Deep Learning, mais de comprendre suffisamment les modèles pour pouvoir les industrialiser.

Page 9 — Phase 6 : Software Engineering

Le MLOps est aussi du Software Engineering appliqué au ML.

- REST et HTTP
- FastAPI
- Validation des données d'entrée
- Tests unitaires et tests d'intégration
- Logging
- Configuration et variables d'environnement
- Packaging Python
- Architecture de projet
- Documentation
- Qualité et maintenabilité du code

Page 10 — Phase 7 : DevOps

- **Docker** : images, containers, Dockerfile, volumes, réseaux et Compose.
- **CI/CD** : automatiser tests, build et déploiement.
- **GitHub Actions** : pipelines automatisés.
- **Réseaux** : HTTP, DNS, ports, reverse proxy, TLS et bases du réseau.
- **Sécurité** : secrets, permissions, dépendances et principe du moindre privilège.

À ce stade, tu dois être capable de transformer une application ML en service reproductible et déployable.

Page 11 — Phase 8 : Cloud

Choisir un Cloud principal et apprendre ses concepts fondamentaux.

- Compute
- Storage
- Databases
- Networking
- IAM
- Container registry
- Monitoring
- Déploiement d'applications
- Gestion des coûts et sécurité

Nous choisirons un fournisseur principal (AWS, Azure ou GCP) et apprendrons ensuite les concepts transposables aux autres.

Page 12 — Phase 9 : MLOps

C'est ici que toutes les compétences précédentes se rejoignent.

- Experiment tracking avec MLflow
- Versioning des données et modèles avec DVC ou équivalent
- Data validation
- ML pipelines
- Model registry
- Model serving
- Déploiement automatisé
- Reproductibilité
- Gestion des versions de modèles
- Rollback et promotion Development → Staging → Production

Page 13 — Phase 10 : Production avancée

- **Kubernetes** : Pods, Deployments, Services, Ingress, ConfigMaps, Secrets, scaling et health checks.
- **Terraform** : Infrastructure as Code.
- **Observabilité** : métriques, logs, traces.
- **Monitoring ML** : performance, data drift et concept drift.
- **Systèmes distribués** : bases utiles pour les architectures à grande échelle.

Page 14 — Phase 11 : LLM / GenAI

Cette spécialisation viendra après les fondamentaux MLOps.

- Transformers
- Hugging Face
- Embeddings
- Vector databases
- RAG
- LLM serving
- Evaluation
- Fine-tuning
- Quantization
- GPU inference
- LLM monitoring

Page 15 — Les langages à maîtriser

Technologie	Objectif dans le parcours
Python	Maîtrise — langage principal pour Data/ML/MLOps
SQL	Solide — bases de données et données
Bash	Solide — Linux et automatisation
YAML	Pratique — CI/CD et Kubernetes
HCL / Terraform	Pratique — Infrastructure as Code
JavaScript / TypeScript	Bases utiles, pas priorité MLOps
C / C++	Important pour la Licence GL et la compréhension bas niveau, mais pas priorité MLOps
Java / C#	Important selon ta Licence et tes projets, mais pas obligatoires pour le MLOps
PHP	Utile selon le cursus/projets, pas nécessaire pour le MLOps
Go	Optionnel, intéressant plus tard pour cloud/infrastructure

Page 16 — Notre organisation pendant la Licence

Formation principale : Python + algorithmique.

Formation parallèle : un seul gros sujet à la fois, en commençant par Git/GitHub.

- Maintenant : Python + Git/GitHub
- Après validation de Git/GitHub : Python + Linux/Bash
- Puis : Python + SQL
- Ensuite : Python + NumPy/Pandas
- Puis : Data + statistiques + ML
- Après : ML + software engineering
- Puis : Docker + CI/CD
- Enfin : Cloud + MLOps + Kubernetes/Terraform

Nous éviterons d'empiler plusieurs technologies lourdes simultanément.

Page 17 — Méthode pédagogique

Chaque grand module aura son propre PDF complet.

- PDF de référence du module
- Cours expliqué page par page
- Exemples
- Exercices progressifs
- Défis sans solution immédiate
- Mini-projets
- Projet final
- Test de validation
- Passage au module suivant seulement lorsque les bases sont suffisamment solides

Exemple : pendant Git, je pourrai te dire « va à la page 14 » et nous travaillerons précisément la partie consacrée aux branches.

Page 18 — Projets et portfolio

Les projets seront progressivement plus proches d'un environnement professionnel.

- Projets Python pour la logique
- Projets utilisant Git/GitHub
- Scripts Linux
- Projets SQL
- Analyse de données
- Modèles ML
- API de prédiction
- Application ML conteneurisée
- Pipeline CI/CD
- Projet MLflow / registry
- Déploiement Cloud
- Projet MLOps complet de bout en bout

Page 19 — Objectif de fin de parcours

À terme, tu dois pouvoir partir d'un problème ML et construire toute la chaîne :

**Données → préparation → entraînement → suivi d'expérience → versioning → validation → registry
→ API → Docker → CI/CD → Cloud → déploiement → monitoring → amélioration**

- Comprendre le code.
- Comprendre l'infrastructure.
- Comprendre le modèle.
- Automatiser les étapes répétitives.
- Diagnostiquer les problèmes.
- Déployer de manière reproductible.
- Maintenir le système en production.

Page 20 — Notre point de départ actuel

Python : nous sommes dans les fondations/intermédiaire.

Nous avons déjà travaillé les variables, conditions, boucles, listes, comprehensions, strings et dictionnaires. Nous avons également commencé à utiliser les fonctions pour résoudre des problèmes.

Prochaine étape Python : continuer les fonctions et consolider la logique, puis progresser vers tuples/sets, exceptions, fichiers, modules, POO et algorithmique plus avancée.

Parallèle actuel : Git/GitHub.

Le premier support complet de ce parcours parallèle est le module « Git & GitHub ».

Page 21 — La règle d'or

Notre objectif n'est pas de collectionner des technologies.

Notre objectif est de **comprendre suffisamment profondément** les concepts pour pouvoir apprendre de nouvelles technologies rapidement et travailler sans dépendre constamment de l'IA.

La progression sera donc :

- **Comprendre** une notion.
- **Pratiquer** avec de petits exercices.
- **Résoudre** un problème sans solution immédiate.
- **Construire** un projet.
- **Expliquer** ce que l'on a fait.
- **Reproduire** la compétence sans aide.
- **Industrialiser** progressivement.

■ Destination : devenir un ingénieur MLOps capable de concevoir, automatiser, déployer et maintenir des systèmes ML en production.